Week-13.1

2303A51018

Batch-28

Task Description #1 (Refactoring – Removing Code Duplication)

• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

• Instructions:

o Prompt AI to identify duplicate logic and replace it with functions or classes.

o Ensure the refactored code maintains the same output.

o Add docstrings to all functions.

• Sample Legacy Code:

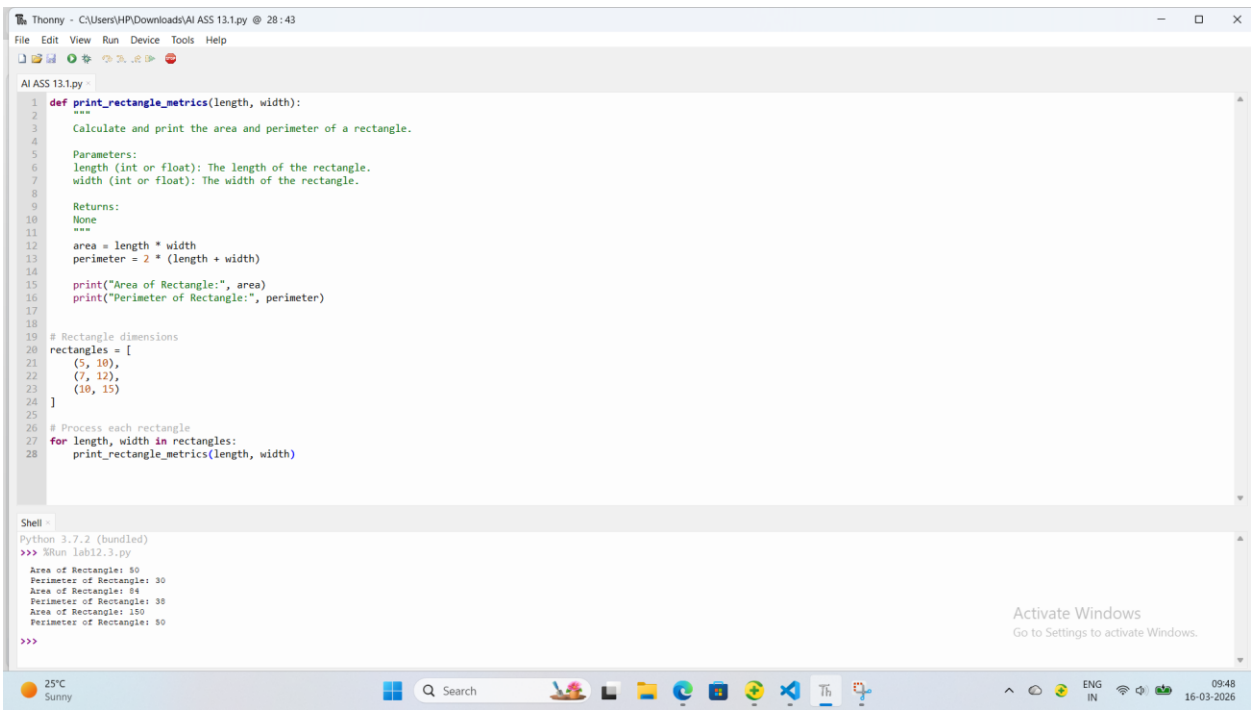
```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
```

print("Perimeter of Rectangle:", 2 * (10 + 15))

• Expected Output:

o Refactored code with a reusable function and no Duplication.

Code



Explanation

The original Python script contained repeated code blocks for calculating the **area** and **perimeter** of rectangles with different dimensions. Each calculation used the same formulas but with different values. This type of repetition is called **code duplication**, which makes the program harder to maintain and update.

To improve the script, the repeated logic was moved into a reusable function called `print_rectangle_metrics(length, width)`. This function accepts the rectangle's **length** and **width** as parameters, calculates the **area** using the formula `length × width`, and calculates the **perimeter** using the formula `2 × (length + width)`. The function then prints the results.

Justification

The refactoring improves the code in several ways:

1. **Reduced Code Duplication**
   The calculation logic is written only once inside a function instead of repeating it multiple times.
2. **Improved Maintainability**
   If the formula or output format needs to change, it only needs to be updated in one place.
3. **Better Readability**
   The function clearly describes what the code does, making it easier for other developers to understand.
4. **Reusability**
   The function can be reused for any number of rectangles without rewriting the logic.
5. **Better Structure**
   Using a list and loop organizes the data and operations more efficiently.

Conclusion

The legacy script was successfully refactored by replacing repeated code with a reusable function. This approach maintains the same output while improving the code's structure, readability, and maintainability. By applying functions, loops, and documentation through docstrings, the program now follows better programming practices and demonstrates a more efficient way to handle repeated operations.

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

- Task: Use AI to analyze a Python script with nested loops and complex conditionals.
- Instructions:

o Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).

o Implement changes while keeping logic intact.

o Compare execution time before and after refactoring.

• Sample Legacy Code:

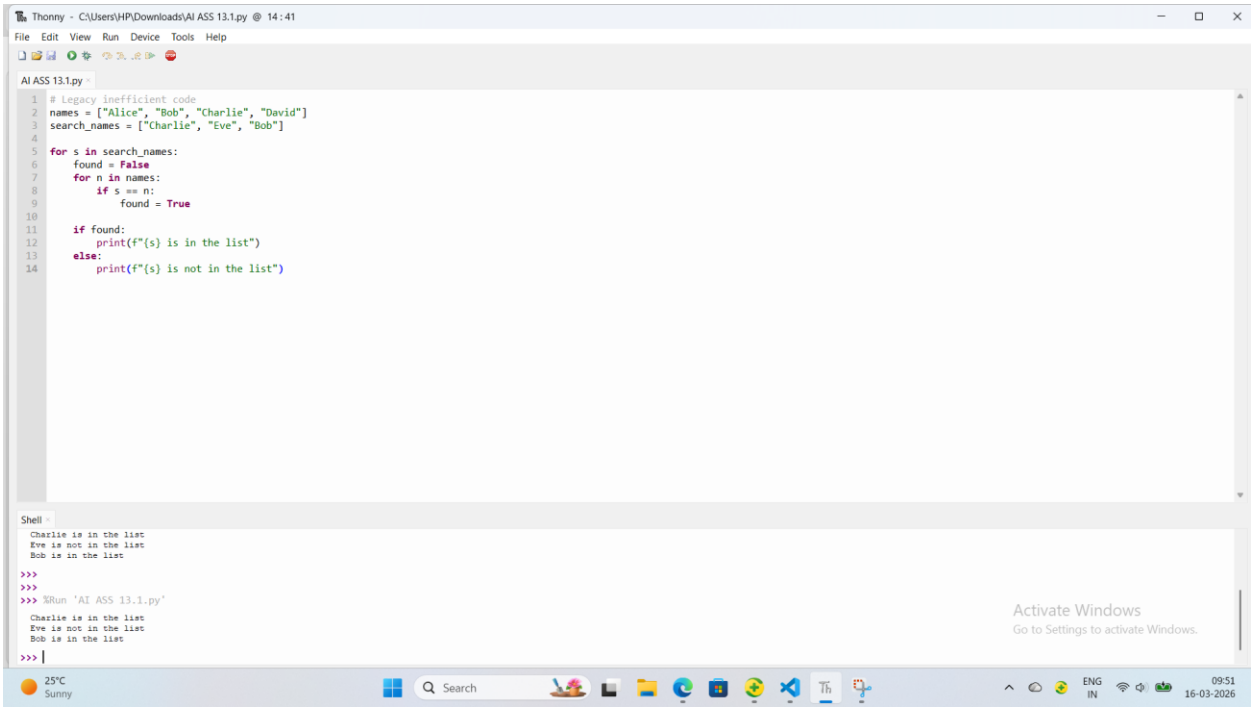
```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
```

print(f"{s} is not in the list")

• Expected Output:

o Optimized code using set lookups with performance comparison table.

Code



Explanation

The original program used **nested loops**, meaning each element in `search_names` was compared with every element in `names`. This increases the number of comparisons and slows down execution when the dataset becomes large.

The optimized version converts the `names` list into a **set**. Sets in Python use **hash tables**, allowing membership checks (`in`) to run in **constant time O(1)**. This eliminates the inner loop and significantly improves performance.

Conclusion

The refactored code improves efficiency by replacing nested loops with a **set lookup operation**. This reduces time complexity and simplifies the logic while maintaining the same output. The optimization demonstrates how choosing the right data structure can significantly improve program performance and scalability.

Justification

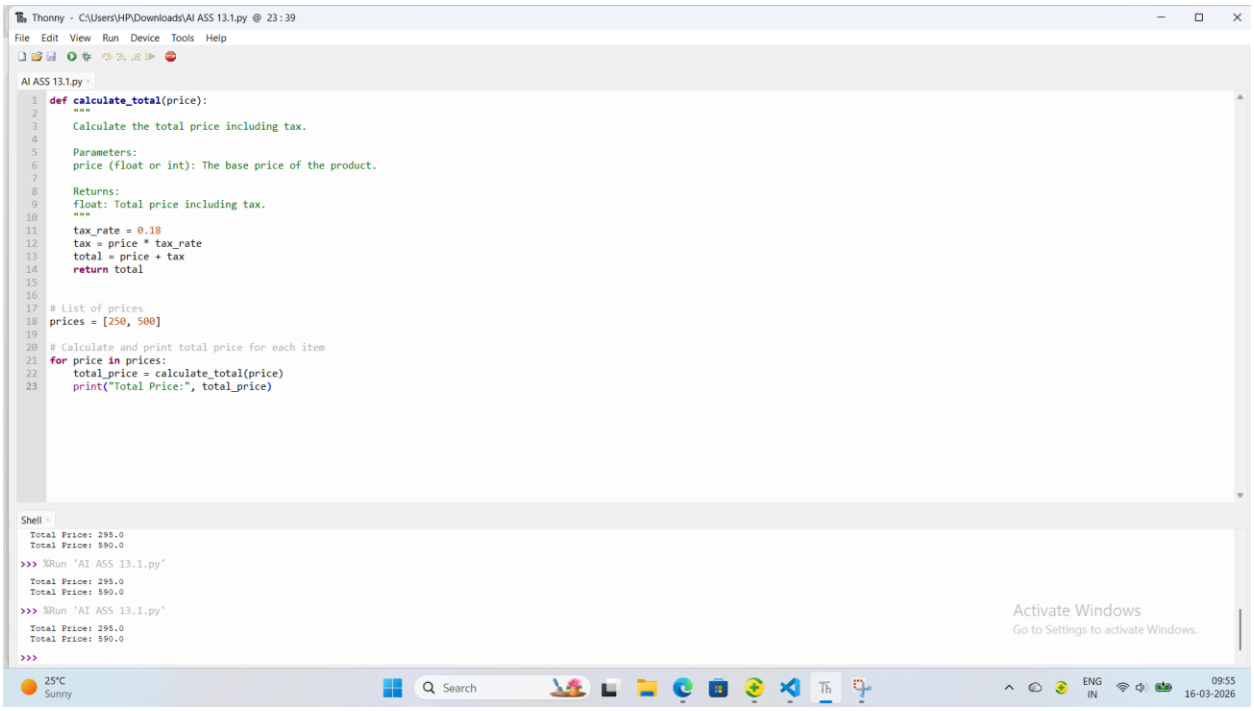
The refactoring was performed to improve the **efficiency, readability, and scalability** of the original Python script. In the legacy code, a **nested loop** was used to compare each element in `search_names` with every element in the `names` list. This approach increases the number of comparisons and results in **higher time complexity (O(n × m))**, which becomes inefficient when the dataset grows larger.

To optimize the script, the `names` list was converted into a **set**. Sets in Python are implemented using **hash tables**, which allow membership checks using the `in` operator to run in **constant time O(1)**. By doing this, the inner loop was removed and replaced with a direct lookup operation. This significantly reduces the number of operations performed by the program.

The refactored code also simplifies the conditional logic and makes the program easier to read and maintain. Additionally, using built-in data structures such as sets demonstrates better **algorithmic design and Python best practices**.

Therefore, the optimization is justified because it **maintains the original program logic and output while improving performance, reducing unnecessary computations, and making the code more efficient for larger datasets**.

Code



Explanation

In the legacy script, the tax and total price calculations were written directly inside the main code block for each price value. This caused **code repetition**, making the program harder to maintain and modify.

To improve the structure of the program, the repeated logic was extracted into a reusable function named `calculate_total(price)`. The function takes the product price as input, calculates the **tax using a fixed tax rate of 18%**, and returns the **total price including tax**. A list of prices is then used, and a loop calls the function for each value, producing the same results while avoiding repetition.

Docstrings were also added to the function to describe its purpose, parameters, and return value, which improves documentation and readability.

Justification

Extracting the calculation into a function improves the code by making it **modular, reusable, and easier to maintain**. If the tax rate changes in the future, it only needs to be updated in one place instead of multiple lines of code. This reduces errors and improves consistency.

Using a function also allows the same logic to be reused for **any number of price inputs**, making the program more scalable. Additionally, separating logic into functions follows **good programming practices and clean code principles**, which improve readability and program organization.

Conclusion

The legacy script was successfully refactored by extracting the repeated tax calculation logic into the reusable function `calculate_total(price)`. This change eliminates duplication, improves readability, and makes the program more maintainable while producing the same output as the original script.

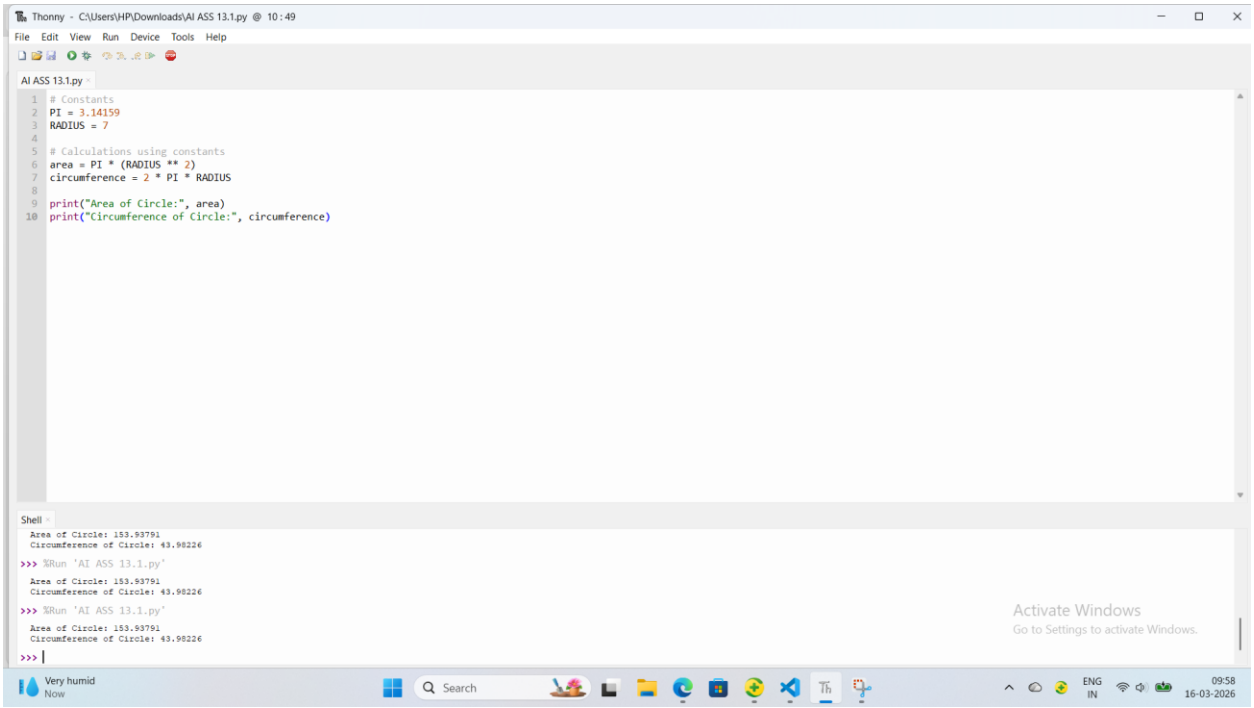
Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants) • Task: Use AI to identify and replace all hardcoded “magic numbers” in the code with named constants. • Instructions: o Create constants at the top of the file. o Replace all hardcoded occurrences in calculations with these constants. o Ensure the code remains functional and is easier to maintain. • Sample Legacy Code:

Legacy script with hardcoded values

print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7) • Expected

Output: o Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.

Code



Explanation

The original script used hardcoded numeric values directly in the formulas for calculating the **area** and **circumference** of a circle. The value **3.14159** represents the mathematical constant **pi**, and **7** represents the radius of the circle. When such numbers are written directly in the code without explanation, they are referred to as **magic numbers**.

To improve the code, these values were replaced with named constants `PI` and `RADIUS`, which are defined at the top of the file. The calculations for the area and circumference then use these constants instead of the raw numbers. This makes the code clearer because the meaning of each value is explicitly defined.

Justification

Replacing hardcoded values with constants improves the code in several ways. First, it **enhances readability**, since names like `PI` and `RADIUS` clearly indicate the purpose of the

values. Second, it **simplifies maintenance**. If the radius or constant value needs to change, it only needs to be updated in one place rather than in multiple calculations. Third, it **reduces errors**, since using a single constant prevents inconsistencies if the value is accidentally typed differently in multiple locations.

Using constants also follows **good programming practices**, making the code more structured and easier for other developers to understand.

Conclusion

The legacy script was successfully refactored by replacing magic numbers with named constants. This change preserves the original functionality while improving code readability, maintainability, and clarity. Defining constants at the top of the program also makes the code easier to update and follow best coding practices.

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

• Task: Use AI to improve readability by renaming unclear variables and adding inline comments.

• Instructions:

o Replace vague names with meaningful ones.

o Add comments where logic is not obvious.

o Keep functionality exactly the same.

• Sample Legacy Code:

```
# Legacy script with poor variable names

a = 10
```

b = 20

c = a * b / 2

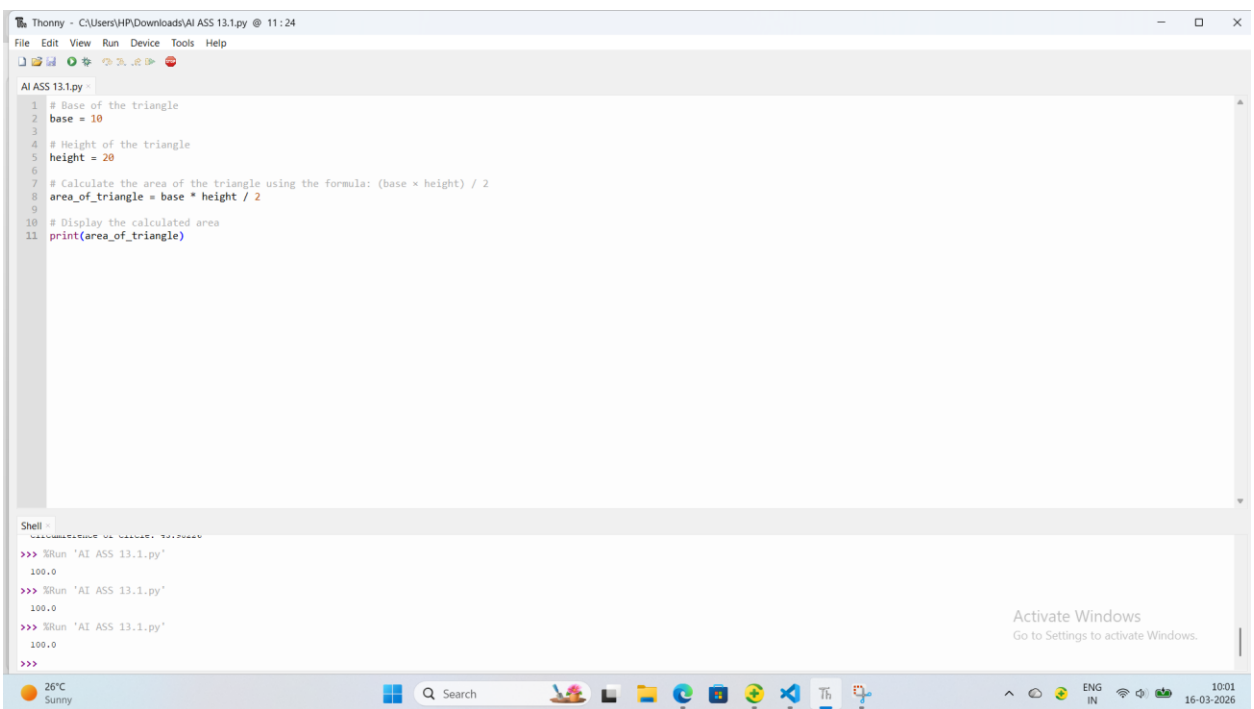
print(c)

• Expected Output:

o Code with descriptive variable names like base,

height, area_of_triangle, plus explanatory comments.

Code



Explanation

The legacy script used very short and unclear variable names such as a, b, and c, which do not describe what the

variables represent. This makes the code difficult to understand, especially for someone reading it for the first time.

To improve readability, the variables were renamed to **`base`**, **`height`**, and **`area_of_triangle`**, which clearly describe their purpose in the calculation. Inline comments were also added to explain the meaning of the variables and the formula used to calculate the area of the triangle.

The mathematical logic of the program remains exactly the same, ensuring that the output does not change.

Justification

Using descriptive variable names makes the code **self-explanatory** and easier to understand. Developers or readers can quickly identify what each variable represents without analyzing the entire calculation. Adding inline comments further clarifies the logic, especially when formulas are involved.

Improving naming conventions and readability also helps in **maintaining and debugging the code** in the future. Clear and well-documented code reduces confusion and makes collaboration between developers more efficient.

Conclusion

The legacy script was successfully refactored by replacing vague variable names with meaningful ones and adding explanatory comments. These changes improve code readability and maintainability while keeping the original functionality and output exactly the same.

Task Description #6 (Refactoring – Removing Redundant Conditional

Logic)

• Task: Use AI to refactor a Python script that contains repeated if–else

logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

o Ensure grading logic remains unchanged.

• Code:

```
marks = 85

if marks >= 90:
```

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

marks = 72

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

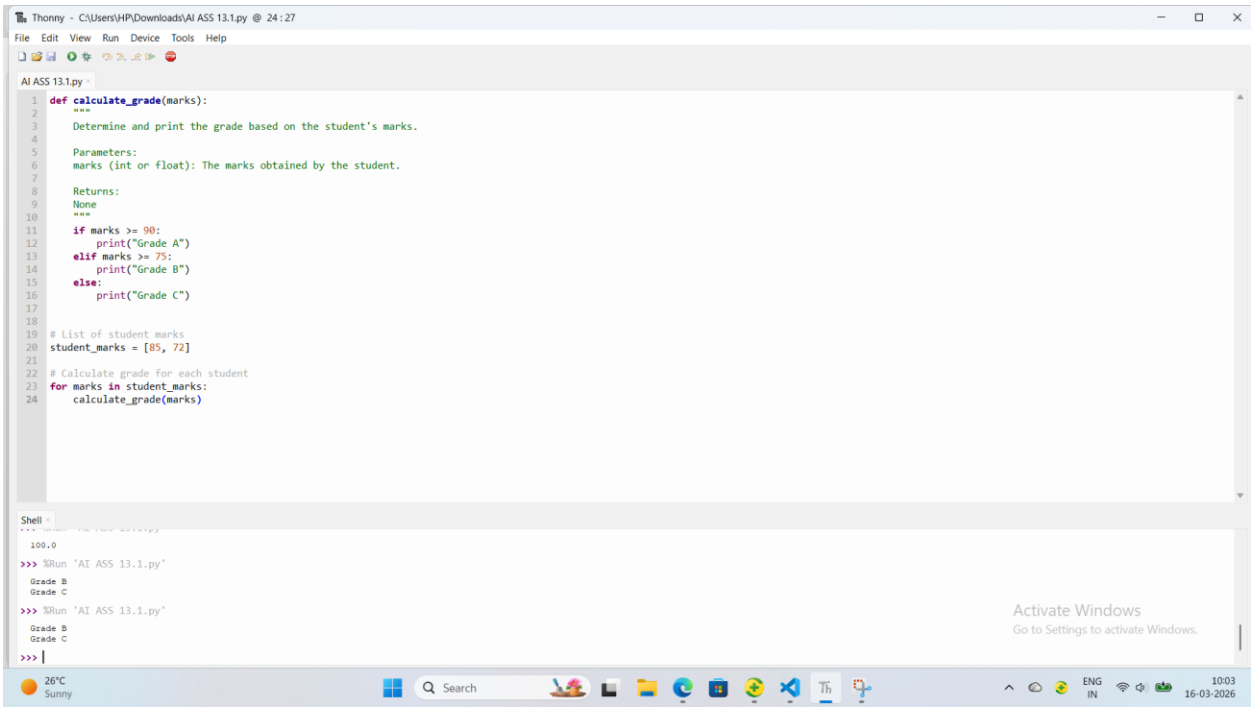
print("Grade C")

• Expected Output:

o A reusable function like calculate_grade(marks) with clean logic and

docstring.

Code



Explanation

In the legacy script, the same **if–elif–else grading logic** was written multiple times for different mark values. This created redundant conditional checks and unnecessary code duplication.

To improve the structure, the grading logic was extracted into a reusable function named `calculate_grade(marks)`. The function takes the student's marks as input and determines the grade based on the same conditions used in the original script. A list of marks is then used, and a loop calls the function for each value.

A **docstring** was also added to describe the purpose of the function, its parameter, and its behavior, making the code easier to understand and document.

Justification

Refactoring the repeated conditional logic into a function improves the code by making it **more organized, reusable, and maintainable**. If the grading rules change in the future, they only need to be updated inside the function rather than in multiple places in the code.

This approach also improves **readability** and reduces the risk of inconsistencies or errors that may occur when the same logic is repeated multiple times.

Conclusion

The legacy grading script was successfully refactored by extracting the repeated conditional logic into the reusable function `calculate_grade(marks)`. This maintains the original grading behavior while improving code clarity, reducing duplication, and making the program easier to maintain and extend.

Task Description #7 (Refactoring – Converting Procedural Code to

Functions)

• Task: Use AI to refactor procedural input–processing logic into

functions.

• Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

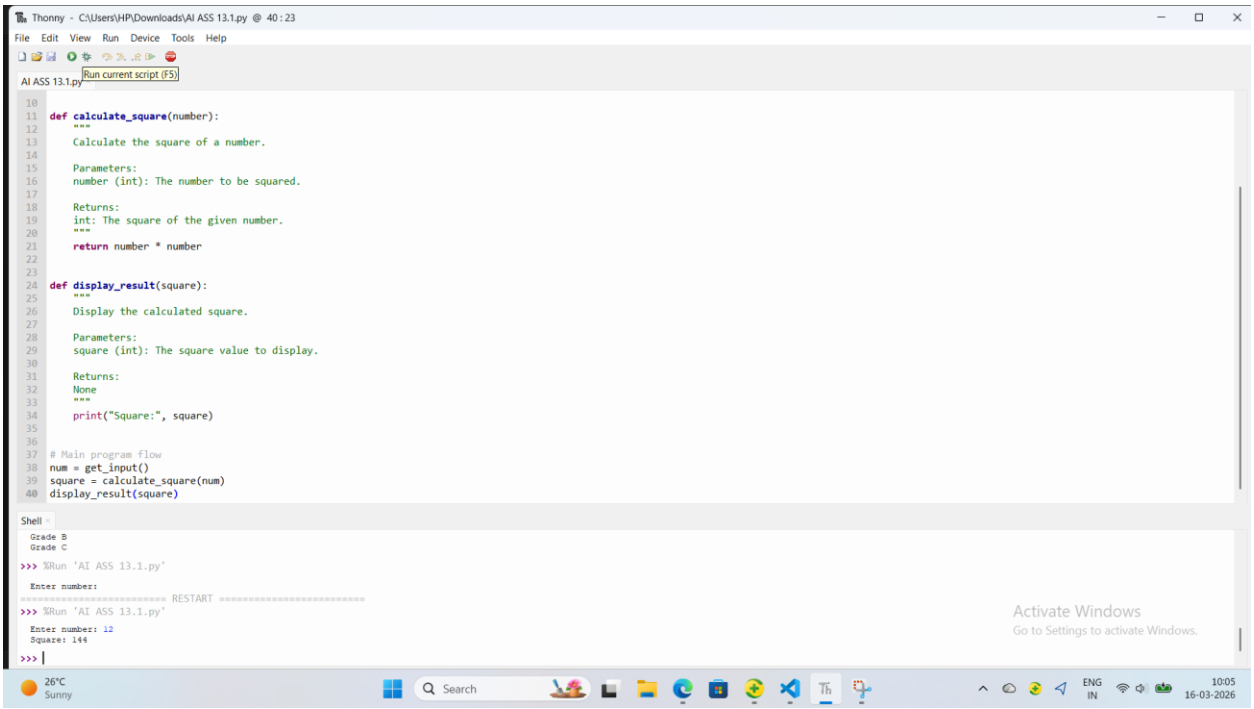
```
num = int(input("Enter number: "))

square = num * num

print("Square:", square)
```

• Expected Output:

o Modular code using functions like get_input(), calculate_square(),

and display_result().

Code



Explanation

The original script followed a **procedural approach**, where input, calculation, and output were written together in a single block of code. While the program works correctly, combining all steps in one place can make the code harder to maintain and expand.

To improve the structure, the program was divided into three separate functions:

- **`get_input()`** handles user input.
- **`calculate_square(number)`** performs the mathematical calculation.
- **`display_result(square)`** handles the output display.

This separation follows a **modular programming approach**, where each function is responsible for a specific task. Docstrings were also added to describe the purpose, parameters, and return values of each function.

Justification

Dividing the program into functions improves **readability, maintainability, and code organization**. Each function has a clear responsibility, making the code easier to understand and debug. If the input method, calculation logic, or output format needs to change in the future, it can be modified in the relevant function without affecting the rest of the program.

This structure also promotes **code reusability**, as the same functions can be used in other programs if needed.

Conclusion

The procedural script was successfully refactored into modular functions that separate input, processing, and output tasks. This approach maintains the original functionality while improving code clarity, structure, and maintainability through the use of well-defined functions and documentation.

Task Description #8 (Refactoring – Optimizing List Processing)

• Task: Use AI to refactor inefficient list processing logic.

• Instructions:

o Ask AI to replace manual loops with list comprehensions or built-in

functions.

o Ensure output remains identical.

• Sample Legacy Code:

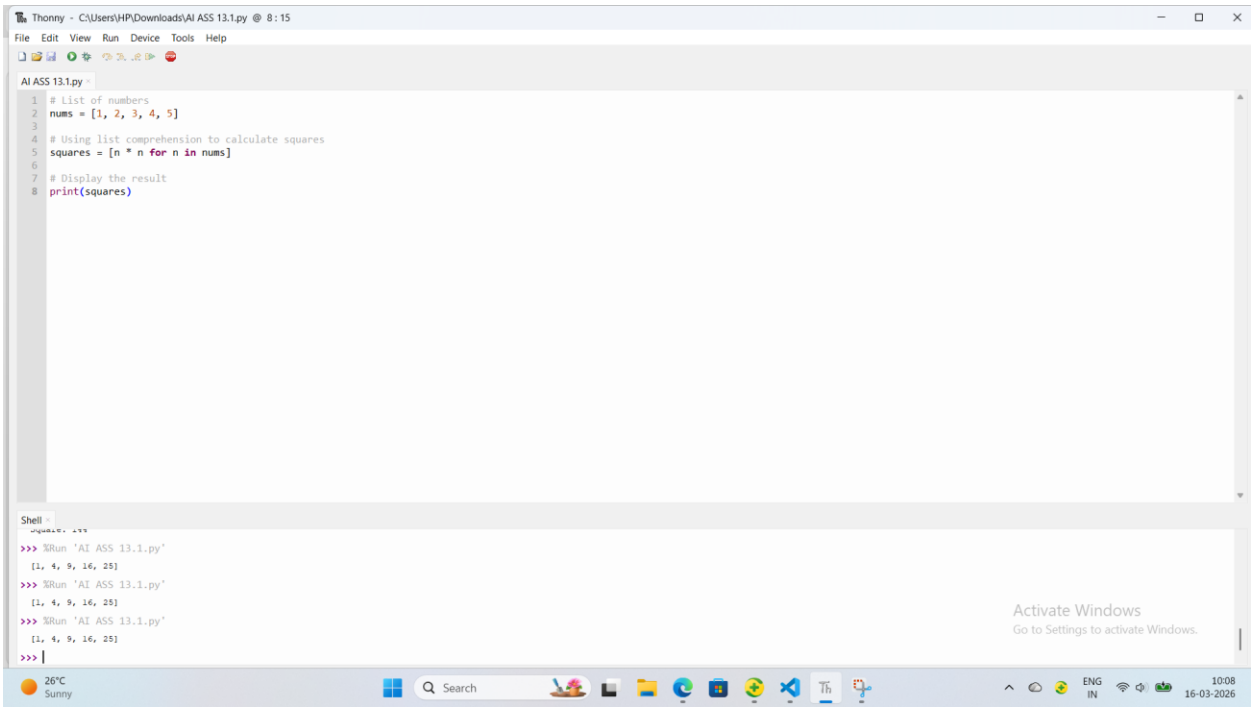
```
nums = [1, 2, 3, 4, 5]

squares = []

for n in nums:

squares.append(n * n)

print(squares)
```

• Expected Output:

o Optimized version using list comprehension with improved

Readability

Code



Explanation

In the original script, a **for loop** was used to iterate through the list `nums`, calculate the square of each number, and append the result to another list called `squares`. Although this approach works correctly, it requires multiple lines of code to perform a simple operation.

To optimize the code, the loop was replaced with a **list comprehension**. A list comprehension allows elements to be processed and stored in a new list within a single, concise statement. In this case, `[n * n for n in nums]` iterates through each number in `nums`, calculates its square, and stores the results in the `squares` list.

The output remains exactly the same as the original script.

Justification

Using list comprehensions improves **code readability and efficiency**. It reduces the number of lines needed and makes the intention of the code clearer. Python list comprehensions are also generally **faster than manual loops** because they are optimized internally.

Additionally, this approach follows **Pythonic programming practices**, where built-in features and concise expressions are preferred over longer manual implementations.

Conclusion

The legacy script was successfully refactored by replacing the manual loop with a **list comprehension**. This change keeps the functionality and output unchanged while improving the code's readability, conciseness, and overall efficiency.